Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Tarih: 23 Ekim 2025 Perşembe. Laboratuvar saatinde yapılan online ek ders. Süre yaklaşık 1 saat 45 dakika.

Genel Konular

- Dizi (string) komutları: `MOVSB/MOVSW`, `CMPSB/CMPSW`, `SCASB/SCASW`, `LODSB/LODSW`, `STOSB/STOSW`.
- Tekrar önekleri (repeat prefix): `REP`, `REPE/REPZ`, `REPNE/REPNZ`.
- Yön bayrağı (direction flag, DF) ile `CLD/STD`.
- İşaret genişletme (sign extension) komutları: `CBW` ve `CWD`.
- Giriş/çıkış komutları `IN` ve `OUT`. İşlemci denetim komutları `HLT`, `NOP`, `WAIT`.
- BCD düzenleme komutları: `AAA`, `AAS`, `AAM`, `AAD`, `DAA`, `DAS`.
- EXE ve COM tipi program yapısı.
- DOSBox içinde MASM, LINK ve DEBUG ile program derleme ve adım adım çalıştırma.
- Örnekler:
  - bir byte içindeki 1'leri sayan program (`BirSay.asm`),
  - word dizide seçmeli sıralama (selection sort, `SelSort.asm`).

Hocanın Özellikle Vurguladığı Kısımlar

- Dizi komutlarında tekrar sayısı `CX`'te tutulur. Her adımda `CX` bir azalır.
- DF = 0 ise adresler artar (ileri). DF = 1 ise adresler azalır (geri).
- Byte komutlarında `SI/DI` 1, word komutlarında 2 değişir.
- `CBW` ve `CWD` işaret bitini dikkate alarak genişletir. Bölme işlemlerinde bu genişletme gerekir.
- EXE programlarda en az bir `FAR` prosedür olmalı. Prosedürün başındaki 5 satır her programda aynıdır.
- `DS`'ye segment adresi doğrudan yüklenemez. Önce `AX`'e alınır, sonra `MOV DS, AX` yapılır.
- Bu dönem yalnız EXE tipi program yazılacak. COM tipine yalnız yapısal fark olarak değinildi.
- Bellek byte adreslemelidir. Word dizide bir sonraki elemana geçmek için indeks 2 artırılır.
- `XCHG` iki bellek gözü arasında çalışmaz. Değerlerden biri önce bir register'a alınır.
- Assembly'de derleyicinin kolaylıkları yoktur. Hangi değişkenin hangi register'da tutulacağı baştan planlanmalı.
- Büyük stack alanı ayırmayın. Küçük örnekler için 15 word civarı yeterli.
- MASM büyük/küçük harf duyarlı (case sensitive) değildir.
- Kodu DEBUG'da `t` ile satır satır çalıştırıp register ve bellek değişimlerini izleyin. Laboratuvar öncesi bu pratiği kazanın.

Kısa Tekrar Notları

- `MOVSB`: `ES:[DI] ← DS:[SI]`. Ardından `SI` ve `DI` ±1.
- `CMPSB`: `DS:[SI] − ES:[DI]` hesaplanır. Sonuç yazılmaz, yalnız bayraklar değişir.
- `SCASB`: `AL − ES:[DI]`. Word sürümü `SCASW` `AX` ile karşılaştırır.
- `LODSB`: `AL ← DS:[SI]`. `STOSB`: `ES:[DI] ← AL`.
- `REP`: `CX` sıfır olana kadar tekrarlar.
- `REPE/REPZ`: `CX ≠ 0` ve `ZF = 1` olduğu sürece tekrarlar. İlk farkta durur.
- `REPNE/REPNZ`: `CX ≠ 0` ve `ZF = 0` olduğu sürece tekrarlar. İlk eşleşmede durur.

- `CBW`: `AL` işaretli olarak `AX`'e genişler. `CWD`: `AX` işaretli olarak `DX:AX`'e genişler.
- `IN AL, 38H` ile 8 bitlik sabit port adresi (0–255) kullanılır. 0–65535 arası port için adres `DX`'e konur: `IN AL, DX`.
- `NOP` gecikme için kullanılabilir. `HLT` işlemciyi kesme gelene kadar durdurur. `WAIT` yardımcı işlemciyi (8087) bekler.
- `AAA`: ASCII toplamı düzeltir. `AAD`: bölmeden önce BCD'yi ikiliye çevirir. `DAA/DAS`: sıkıştırılmış BCD (packed BCD) toplama/çıkarma sonucunu düzeltir.
- EXE: ayrı stack, data ve code segmentleri, `FAR` prosedür, `RETF`.
- COM: tek segment, `ORG 100H`, `NEAR` prosedür, `RET`, en fazla 64 KB.
- DEBUG komutları:
  - `u`: makine kodunu komutlara çevirip gösterir (unassemble).
  - `t`: tek adım çalıştırır (trace).
  - `r`: register'ları gösterir.
  - `d ds:0`: data segmentin 0 adresinden belleği gösterir (dump).
  - `g`: programı sonuna kadar çalıştırır.
  - `q`: DEBUG'dan çıkar.

Detaylı Açıklamalar

Dizi komutları ve yön bayrağı

- Dizi komutları ardışık bellek gözleri üzerinde tekrarlı işlem için tasarlanmıştır.
- 8086'da bellek 1 MB'tır ve 20 bit ile adreslenir. Kodda görülen 16 bitlik adresler, segment register'ının gösterdiği alan içindeki ofsettir.
- Kaynak her zaman `DS:SI`, hedef her zaman `ES:DI` olur.
- `CLD` DF'yi 0 yapar ve adresler artar. `STD` DF'yi 1 yapar ve adresler azalır.

Dizi kopyalama (slayt):

```
LEA SI, dizi1
LEA DI, dizi2
CLD
MOV CX, size
REP MOVSB        ; dizi2 <- dizi1
```

Diziyi sıfırlama (`STOSB` ile):

```
LEA DI, dizi
CLD
XOR AL, AL       ; AL = 0
MOV CX, size
REP STOSB        ; dizinin her byte'ına 0 yazılır
```

Örnek 1: `REP MOVSB` izlemesi

- `address1` = 0000H. Buradaki byte'lar sırayla 41H, 50H, 70H, 4AH.
- `address2` = 2000H. Kopyanın hedefi.
- Kod: `LEA SI, address1`, `LEA DI, address2`, `MOV CX, 4`, `CLD`, `REP MOVSB`.

Adım	Kopyalanan	SI sonra	DI sonra	CX sonra
1	41H → 2000H	0001H	2001H	3
2	50H → 2001H	0002H	2002H	2
3	70H → 2002H	0003H	2003H	1
4	4AH → 2003H	0004H	2004H	0

Örnek 2: Parola karşılaştırma (`REPE CMPSB`)

- `dizi1` = "password01", `dizi2` = "pasword123". İkisi de 10 karakter.

```
LEA SI, dizi1
LEA DI, dizi2
```

```
MOV CX, 10
REPE CMPSB
```

- p, a, s eşleşir. 4. karakterde “s” ile “w” farklıdır.
- Döngü 4. karşılaştırmadan sonra durur. Çıkışta `CX` = 6 ve `ZF` = 0 olur.

Örnek 3: E-posta adresinde “@” arama (`REPNE SCASB`)

- “@” karakterinin ASCII kodu 40H.
- `email` dizisi 0200H’den başlar ve 10 byte’tır. 40H değeri 0205H adresindedir.

```
LEA DI, email
MOV CX, 10
CLD
MOV AL, '@'
REPNE SCASB      ; AL ile ES:[DI] eşleşene kadar ara
```

- Eşleşme 6. karşılaştırmada olur. Çıkışta `ZF` = 1, `CX` = 4, `DI` = 0206H.
- Karakter bulunmazsa `CX` = 0 ve `ZF` = 0 ile çıkar.

`CBW` ve `CWD`

- `AL`’nin en yüksek anlamlı biti 1 ise sayı negatiftir.
- `CBW` bu işaret bitini `AH`’nin tüm bitlerine yayar. Örnek: `AL` = F0H ise `AX` = FFF0H olur.
- `CWD` aynı işi `AX`’ten `DX:AX`’e yapar. 16 bitlik bölmede bölünen `DX:AX`’te olduğu için gerekir.

Giriş/çıkış ve denetim komutları

- Klavye, ekran, fare gibi çevre birimleri bir arayüz (interface) üzerinden bağlanır. Arayüzdeki register’lara port denir.
- `IN` porttan akümülatöre (`AL` veya `AX`) okur. `OUT` akümülatörden porta yazar.
- Bu komutların uygulaması vizeden sonraki konularda yapılacak.

BCD düzenleme komutları (slayt örnekleri)

```
XOR AX, AX
MOV AL, '6'      ; 36H
ADD AL, '7'      ; AL = 6DH
AAA              ; AX = 0103H, BCD olarak 13

MOV AH, 04H
MOV AL, 05H      ; AX'te BCD 45
AAD              ; AX = 002DH (45 ondalık)

MOV AL, 75H
ADD AL, 19H      ; AL = 8EH
DAA              ; AL = 94H (75 + 19 = 94)

MOV AL, 42H
SUB AL, 13H      ; AL = 2FH
DAS              ; AL = 29H (42 - 13 = 29)
```

- BCD gösterim, 7 segment gösterge (display) gibi birimlere veri gönderirken gerekir.
- `DAA` yalnız `AL` üzerinde çalışır. Toplamdan hemen sonra kullanılır.

EXE tipi program iskeleti

```
STACKSG SEGMENT PARA STACK 'STACK'
        DW 32 DUP(?)
STACKSG ENDS

DATASG  SEGMENT PARA 'DATA'
```

```
SAYI    DB ?
ELEMAN  DW ?
DATASG  ENDS

CODESG  SEGMENT PARA 'CODE'
        ASSUME CS:CODESG, DS:DATASG, SS:STACKSG
BASLA   PROC FAR
        PUSH DS
        XOR AX, AX
        PUSH AX
        MOV AX, DATASG
        MOV DS, AX
        ; kendi kodunuz buraya
        RETF
BASLA   ENDP
CODESG  ENDS
        END BASLA
```

- `PARA`, segmentin 16'ya bölünebilen (paragraf sınırı) bir adreste başlamasını sağlar.
- `DW 32 DUP(?)`, 32 word'lük (64 byte) stack ayırır.
- İlk üç satır dönüş adresini (`DS:0000`) stack'e koyar. `RETF` programı bu adrese döndürür ve program DOS'a geri döner.
- Sonraki iki satır programın kendi data segmentini `DS`'ye yükler.
- `END BASLA`, EXE başlatılınca çalışacak prosedürü belirtir.

COM tipi program iskeleti

```
CODESG  SEGMENT PARA 'CODE'
        ORG 100H
        ASSUME CS:CODESG, SS:CODESG, DS:CODESG, ES:CODESG
BASLA   PROC NEAR
        ; kod
        RET
BASLA   ENDP
SAYIB   DB ?
SAYIW   DW ?
CODESG  ENDS
        END BASLA
```

- Data, stack ve code aynı segmenttedir. Değişkenler code segment içinde tanımlanır.

Örnek 4: Bir byte'taki 1'leri sayma (`BirSay.asm`)

```
stacksg SEGMENT PARA 'yigin'
        DW 15 DUP(?)
stacksg ENDS
datasg  SEGMENT PARA 'veri'
sayi    DB 55H
datasg  ENDS

codesg  SEGMENT PARA 'kod'
        ASSUME CS:codesg, DS:datasg, SS:stacksg
SAY     PROC FAR
        PUSH DS
        XOR AX,AX
        PUSH AX
        MOV AX,datasg
        MOV DS,AX

        XOR BL,BL       ; Reset BL
        MOV CX,8
```

```
        MOV AH,sayi
L1:     SHL AH,1
        JNC SON
        INC BL
SON:    LOOP L1
        RETF
SAY     ENDP
codesg  ENDS
        END SAY
```

- Derleme ve çalıştırma DOSBox içinde yapılır:
  1. `masm birsay.asm;` ile `.obj` dosyası üretilir.
  2. `link birsay.obj;` ile `.exe` dosyası üretilir.
  3. `debug birsay.exe` ile program açılır.
- DEBUG izlemesi:
  - `MOV AH,[0000]` sonrası `AX` = 076CH'den 556CH'ye geçer (`DS:0000` = 55H).
  - `SHL AH,1` sonrası `AX` = AA6CH olur. Sola kaydırma 2 ile çarpmadır: 55H × 2 = AAH.
  - 55H = 0101 0101b. İlk kaydırmada dışarı çıkan bit 0'dır, `CF` = 0 ve `BL` artmaz.
  - 8 adım sonunda `BL` = 4 olur (55H'de dört tane 1 var).

Örnek 5: Word dizide seçmeli sıralama (`SelSort.asm`)

Akış: `i = 0 … n−2` için `min = i`. `j = i+1 … n−1` için `dizi[j] < dizi[min]` ise `min = j`. Sonra `dizi[i]` ile `dizi[min]` yer değiştirir.

Register planı: - `SI`: i (byte ofseti). - `DI`: j (byte ofseti). - `BX`: min (byte ofseti). - `CX`: döngü sayacı.

```
STEK    SEGMENT PARA STACK 'STK'
        DW 30 DUP(?)
STEK    ENDS

DSG     SEGMENT PARA 'DTS'
n       DW 10
Dizi    DW 11,2,80,20,25,1,3,7,8,9
DSG     ENDS

CSG     SEGMENT PARA 'CODE'
        ASSUME CS:CSG, DS:DSG, SS:STEK
BASLA   PROC FAR
        PUSH DS
        XOR AX,AX
        PUSH AX
        MOV AX, DSG
        MOV DS, AX
;kod buradan sonra
        XOR SI,SI
        MOV CX,n
        DEC CX              ; dış döngü n-1 kez
disDon: MOV BX,SI           ; min = i
        PUSH CX             ; dış sayacı sakla
        MOV DI,SI
        ADD DI,2            ; j = i+1 (word: +2 byte)
        MOV DX,DI
        SHR DX,1            ; DX = j'nin eleman indeksi
        MOV CX,n
        SUB CX,DX           ; iç döngü n-(i+1) kez
icDon:  MOV AX,dizi[DI]
        CMP AX,dizi[BX]
        JGE falseK
        MOV BX,DI           ; min = j
falseK: ADD DI,2
```

```
        LOOP icDon
        POP CX
        MOV AX, dizi[SI]    ; XCHG bellek-bellek çalışmaz
        XCHG AX, dizi[BX]
        MOV dizi[SI],AX
        ADD SI,2
        LOOP disDon
        RETF
BASLA   ENDP
CSG     ENDS
        END BASLA
```

- Son üç satır (`LOOP disDon` ve sonrası) ekranda tam görünmüyor. Akış diyagramına ve hocanın anlatımına göre tamamlandı.
- Aynı mantık Python ile çalıştırıldı. Sonuç: `1, 2, 3, 7, 8, 9, 11, 20, 25, 80`.
- `JGE` işaretli karşılaştırmadır. Dizide negatif sayı olursa da doğru sıralar.
- `n` ve dizi elemanları `DW` ile tanımlandı. Böylece 255'ten büyük değerler ve uzun diziler kullanılabilir.
- Sıralamayı izlemek için DEBUG'da `d ds:0` ile data segmenti görüntüleyin. En küçük eleman her dış turda dizinin başına yerleşir.

Ders sonrası notlar

- Hoca bir yerde "ekstra segment belirtilmediyse data segment olarak bulunur" dedi. Bu doğru değil. `ES` otomatik olarak `DS`'ye eşitlenmez.
  - DEBUG ekranında da `DS` = 076C, `ES` = 075A (PSP) görünüyor.
  - Dizi komutları hedefi her zaman `ES:DI`'dan alır. EXE programda bu komutları kullanmadan önce `MOV ES, AX` ile `ES`'yi de data segmente yükleyin.
- Tablette `MOVSB` örneğinin ikinci satırı `LEA SI, address2` yazılmış. Doğrusu `LEA DI, address2`. Hoca anlatırken de `DI` diyor.
- `REP` örneğinde hoca bir yerde "CX register'ını bir artırarak" dedi. `CX` her adımda bir azalır.
- Altyazıda "65536 port için port numarasının DX'te olması gerekmiyor" geçiyor. Doğrusu: 255'ten büyük port adresleri için port numarası `DX`'te olmalıdır.
- `DAA` açıklamasında hoca "9E olarak görünür" dedi. Slayttaki ve doğru değer 8EH (75H + 19H).
- `CMPSB` sonrası öğrenci sorusunda, eşitliğin `CX` ile kontrol edileceği onaylandı. Güvenli kontrol `ZF` ile yapılır (`JE`/`JNE`). Fark son karakterde çıkarsa `CX` yine 0 olur, ama `ZF` = 0 kalır.
- `BirSay.asm` linklenirken `warning L4021: no stack segment` uyarısı çıktı. Hoca bunun önemsiz olduğunu söyledi.
  - Uyarının nedeni stack segment tanımında `STACK` birleştirme türünün olmaması (`SEGMENT PARA 'yigin'`).
  - Slayttaki gibi `SEGMENT PARA STACK 'STACK'` yazılırsa uyarı çıkmaz ve `SS:SP` programın kendi stack'ini gösterir.
- Hoca "Q ile programı sonuna kadar çalıştırdık" dedi. DEBUG'da `q` çıkış komutudur. Programı sonuna kadar çalıştırmak için `g` kullanılır.
- `SI` ve `DI` için "dizinin indeksi" denildi. Bunlar `dizi` etiketine göre byte ofsetidir. Bu yüzden word dizide 2 artırılır. Eleman indeksi `SHR DX,1` ile bulunur.
- Altyazıda "SMB", "SM", "SME" geçen yerler ASM (assembly) anlamındadır. "alt komutu" yerleri bağlama göre `OUT` ve `HLT`'dir.

Duyurular

- Ders duvarında paylaşılan dosyalar:
  - MASM, LINK ve DEBUG'ı içeren 8086 ASM/debug paketi,
  - boş EXE iskeleti (`BosAsm.zip`),
  - 1'leri sayan program.
- `SelSort.asm` de ders duvarına konacak.
- Ertesi gün (24 Ekim 2025) yüz yüze derste bu konular tahtada tekrar edilecek. `SelSort` DEBUG'da çalıştırılıp bellek değişimleri gösterilecek.

- Bu Cuma laboratuvarla ilgili soru verilecek. Önümüzdeki haftalarda laboratuvar çalışmaları başlıyor. Kod yazıp DEBUG ile denemeye başlayın.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.